



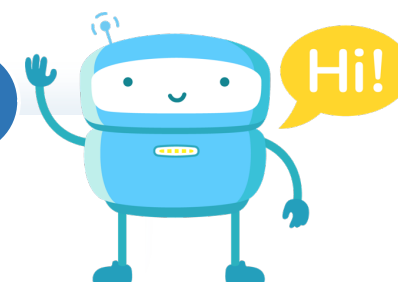
自然语言处理

第三章（三）：神经网络基础

—RNN和LSTM

注意：本PPT仅供个人学习使用，任何组织和个人不得公开传播或用于任何商业盈利用途，否则一切后果由该组织或个人承担。

授课老师： 缪尧



应用实例

- 槽填充 (Slot Filling)



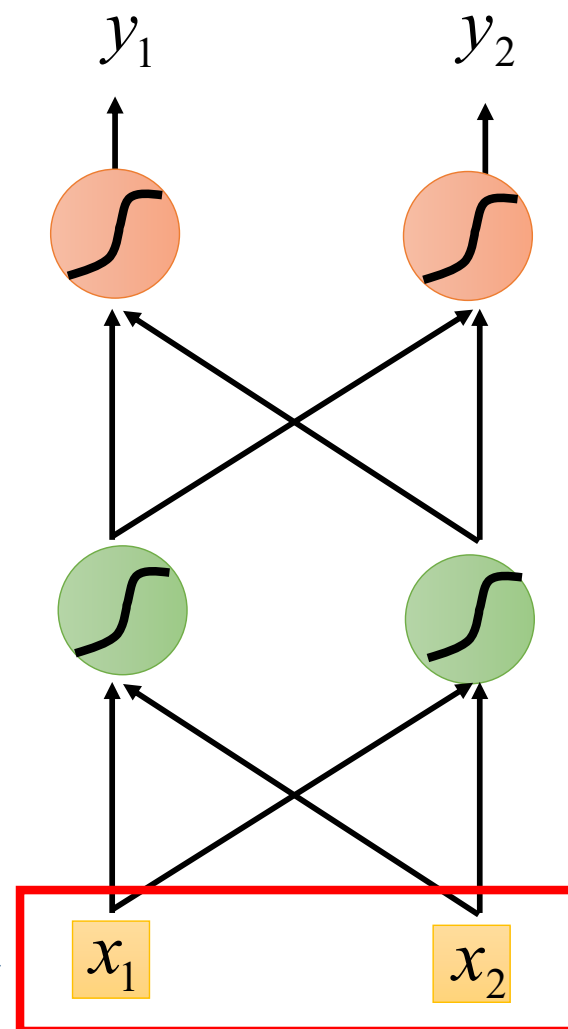
应用实例

通过前馈神经网络解决槽
填充问题?

Input: a word

(每个词都被表示为一个向量)

Shenzhen



1-of-N encoding

如何将每个词表示为向量?

1-of-N Encoding 词汇表 = {apple, bag, cat, dog, elephant}

这个向量的维度是词汇表的大小。

apple = [1 0 0 0 0]

每个维度对应词汇表中的一个词。

bag = [0 1 0 0 0]

cat = [0 0 1 0 0]

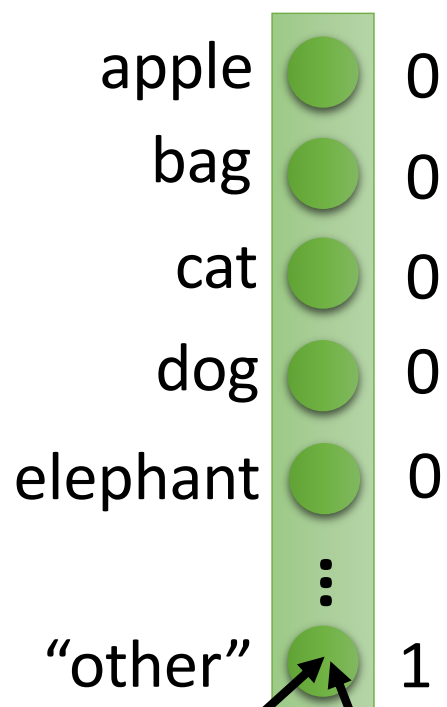
对于这个词的维度是1，其他维度的值是0。

dog = [0 0 0 1 0]

elephant = [0 0 0 0 1]

Beyond 1-of-N encoding

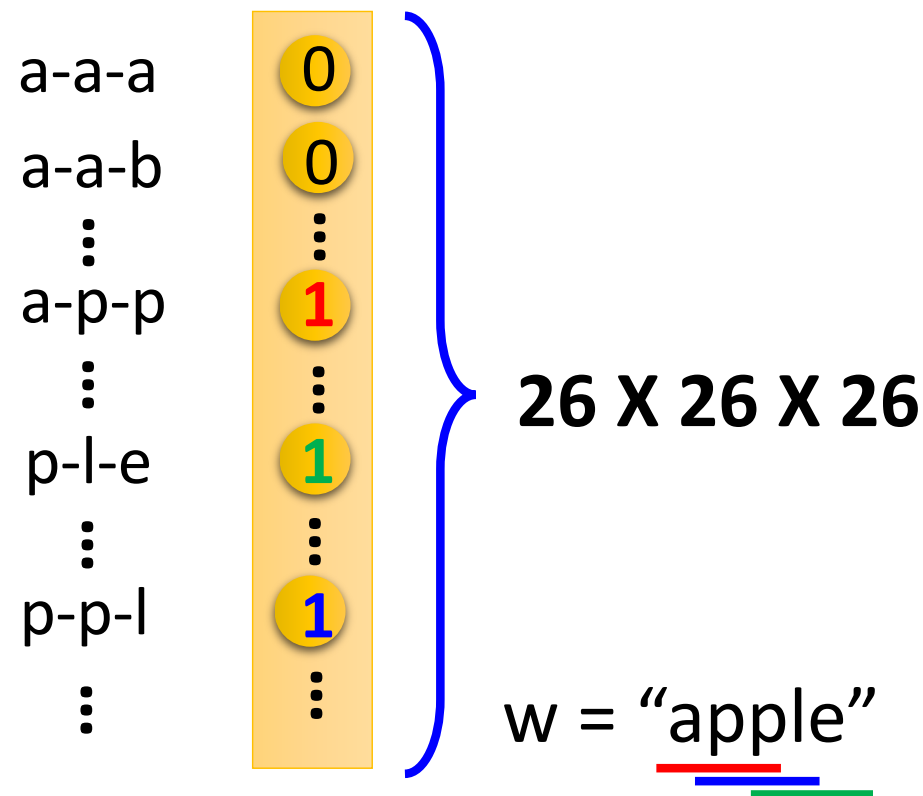
“Other”的维度



w = “Gandalf”

w = “Sauron”

Word hashing



应用实例

通过前馈神经网络解决槽填充问题?

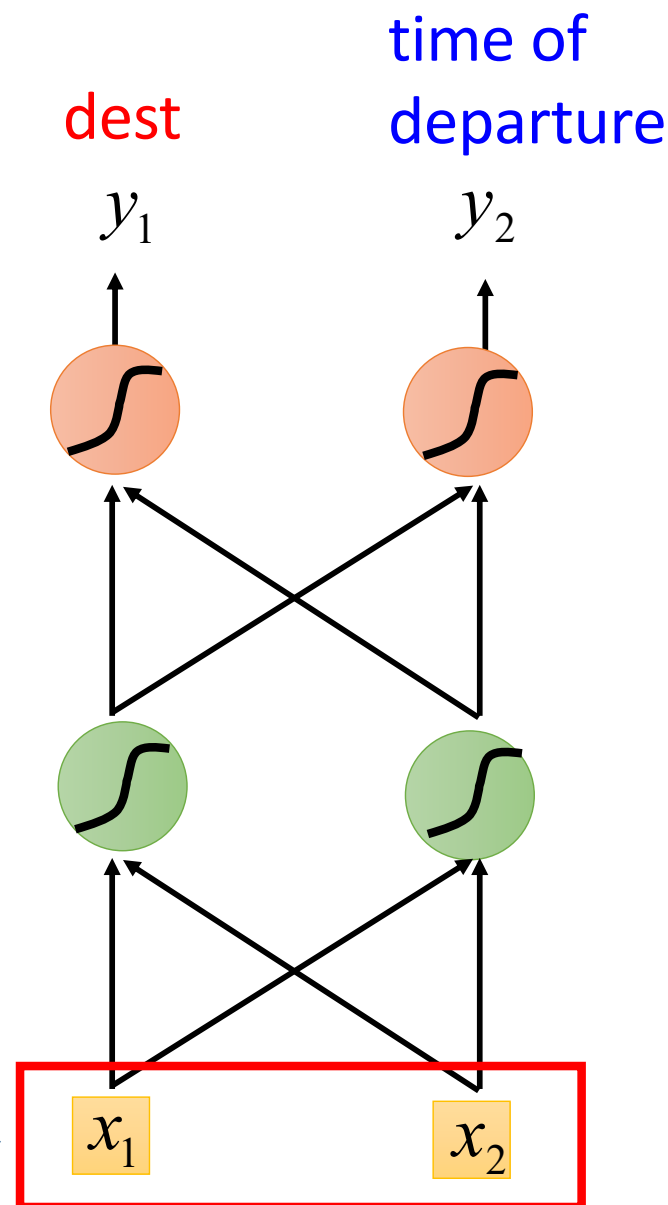
Input: a word

(每个词都被表示为一个向量)

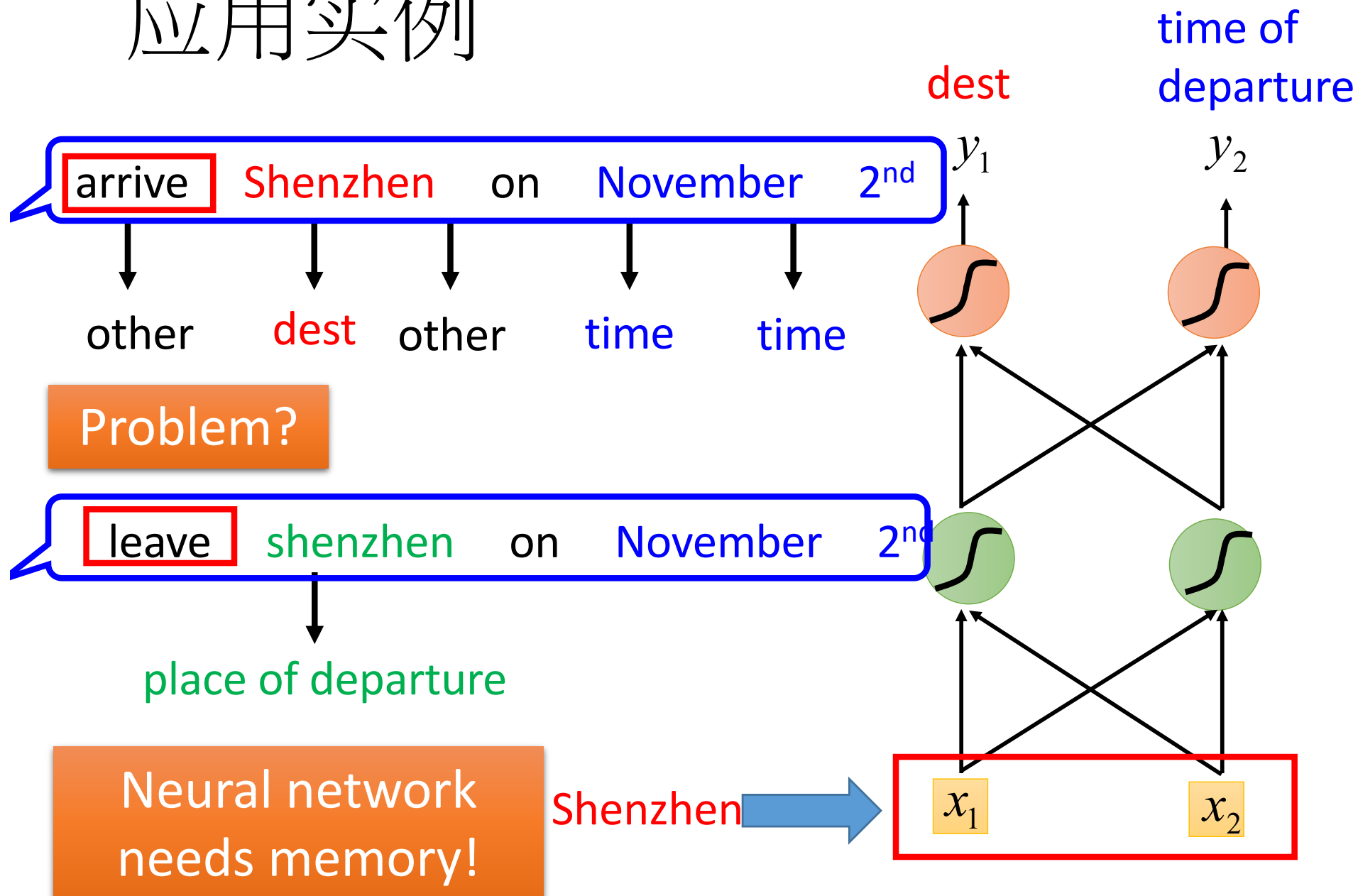
Output:

输入词属于各个槽位 (slots) 的概率分布

Shenzhen

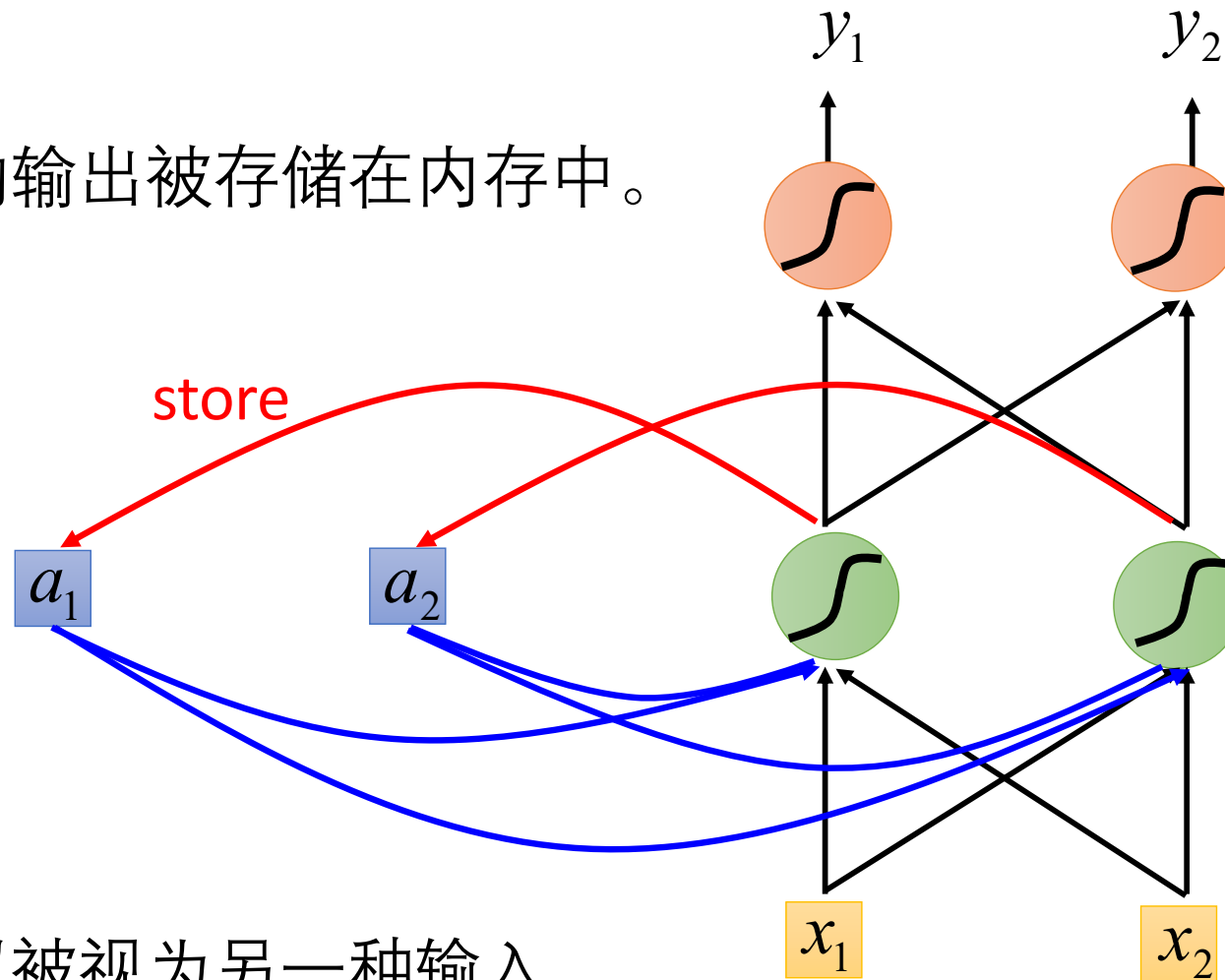


应用实例



Recurrent Neural Network (RNN)

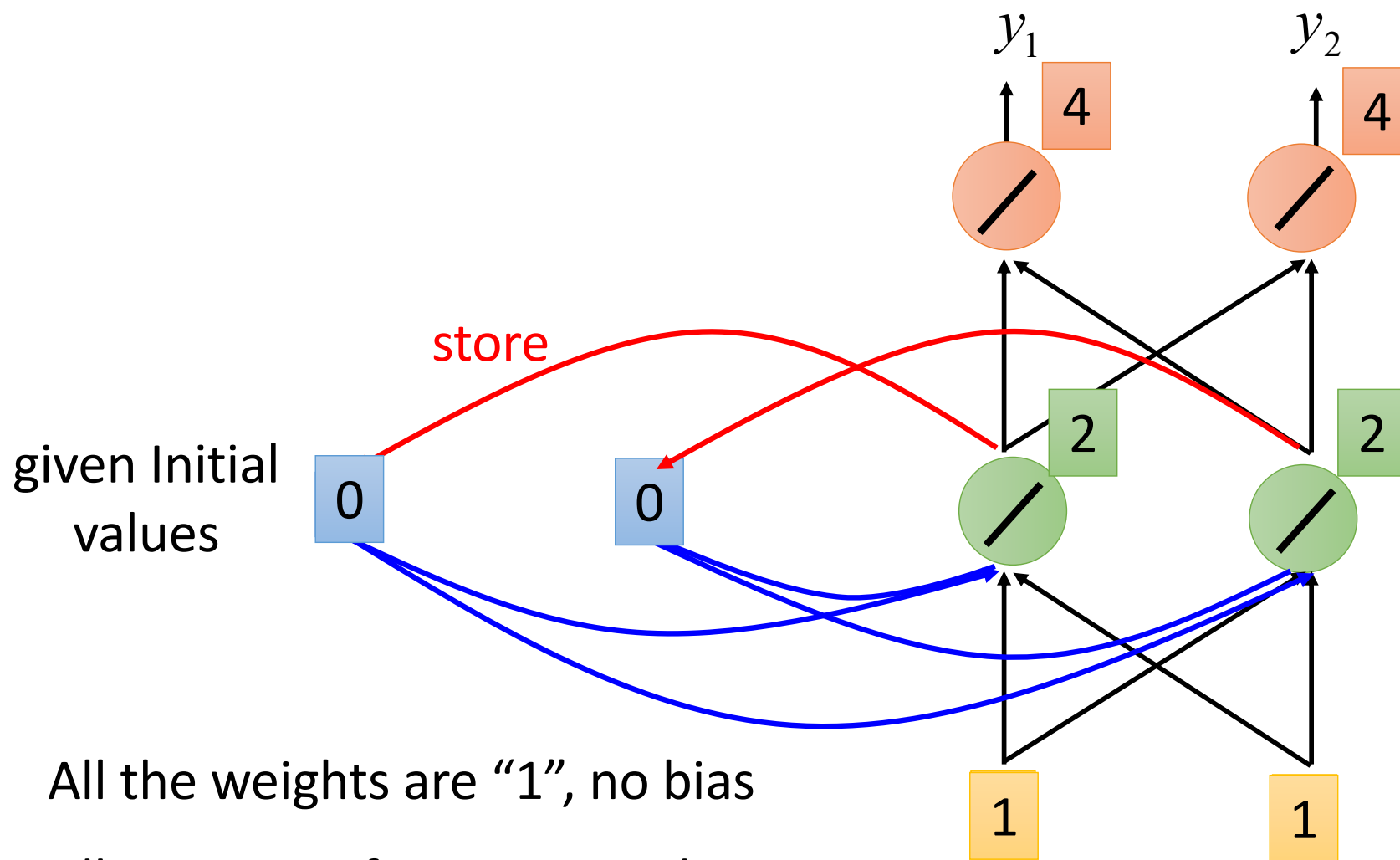
隐藏层的输出被存储在内存中。



内存可以被视为另一种输入。

实例

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots\dots$
output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$

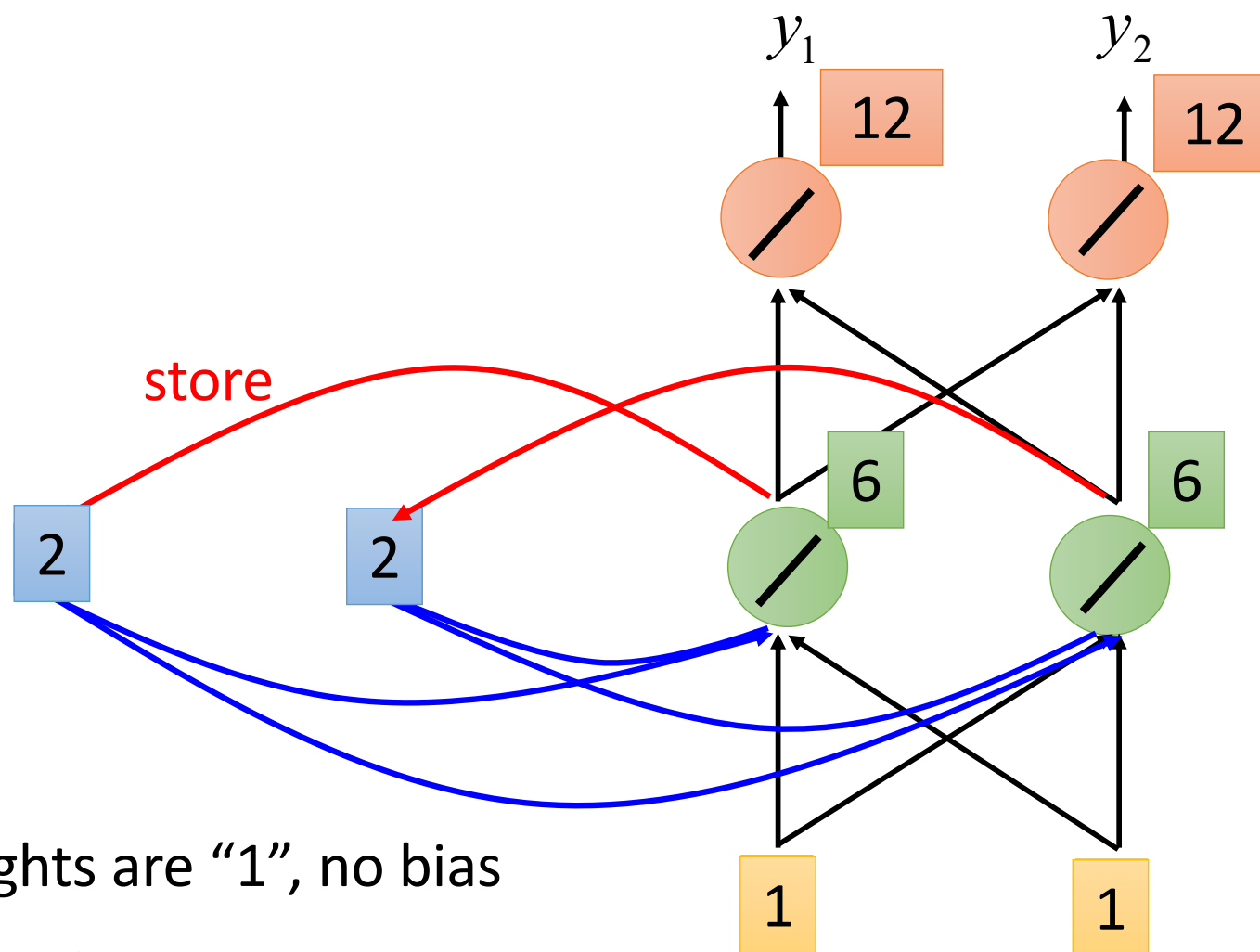


All the weights are “1”, no bias

All activation functions are linear

实例

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots \dots$
output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix}$



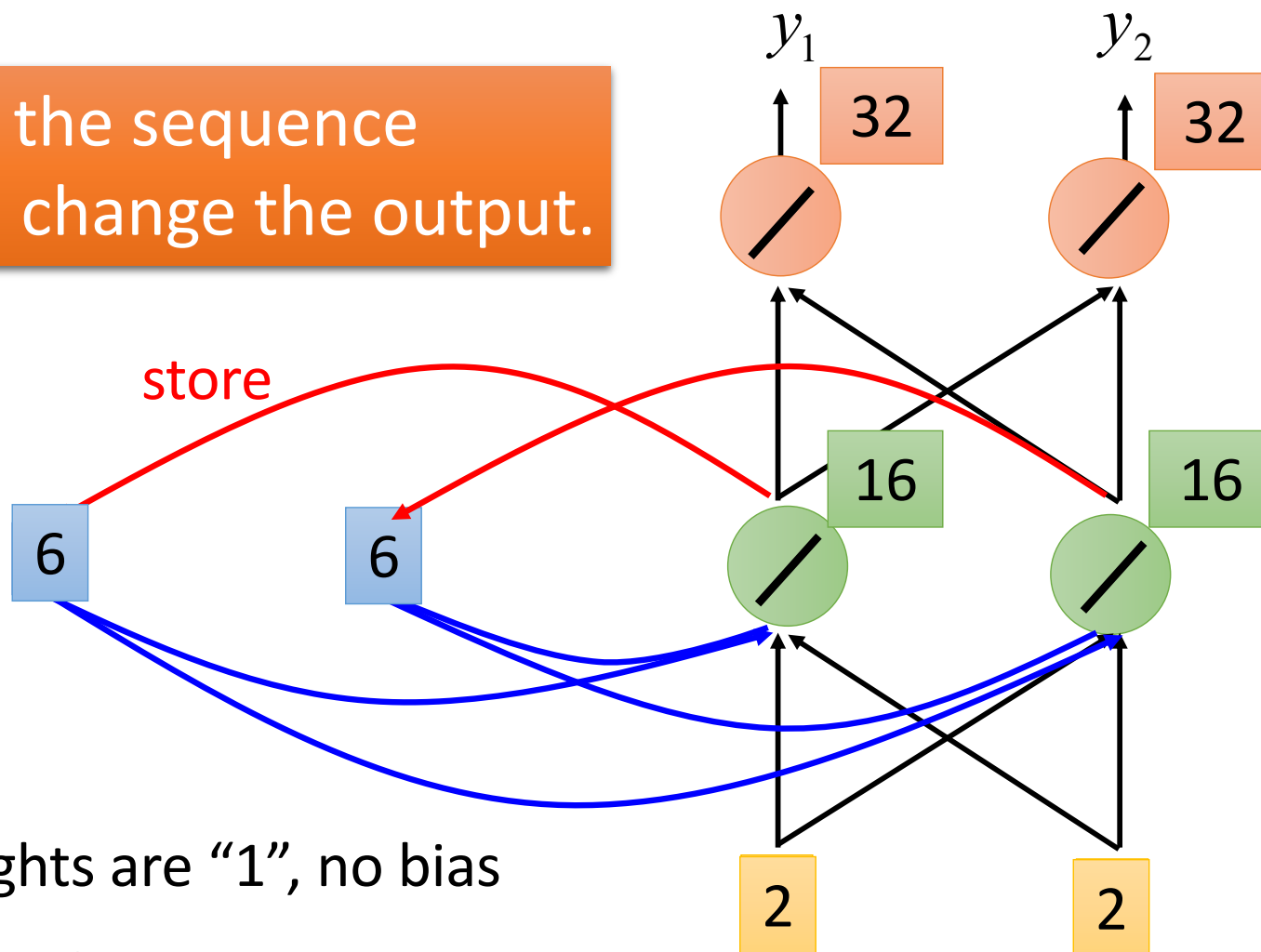
All the weights are “1”, no bias

All activation functions are linear

实例

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots \dots$
output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix} \begin{bmatrix} 32 \\ 32 \end{bmatrix}$

Changing the sequence order will change the output.



All the weights are “1”, no bias

All activation functions are linear

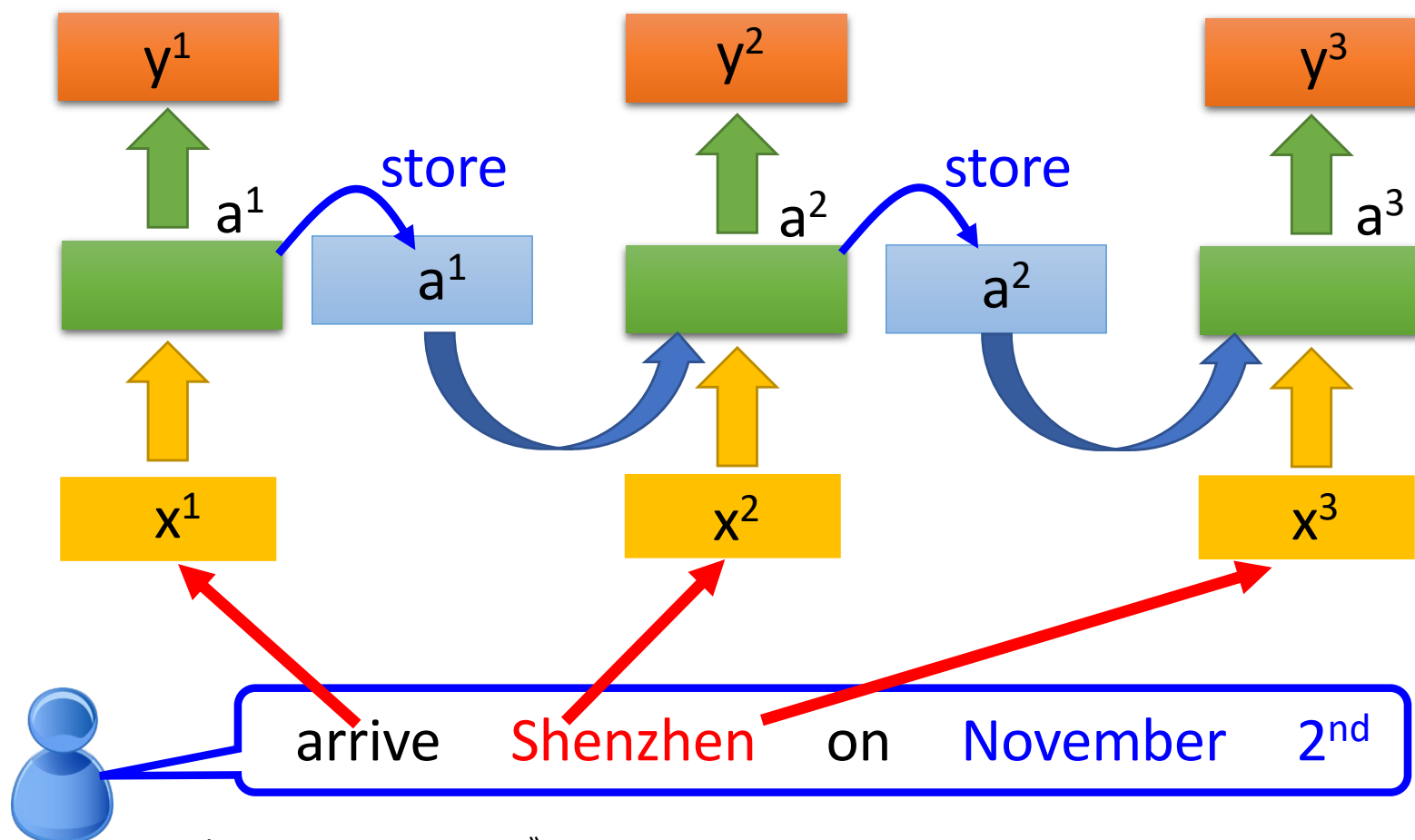
RNN

The same network is used again and again.

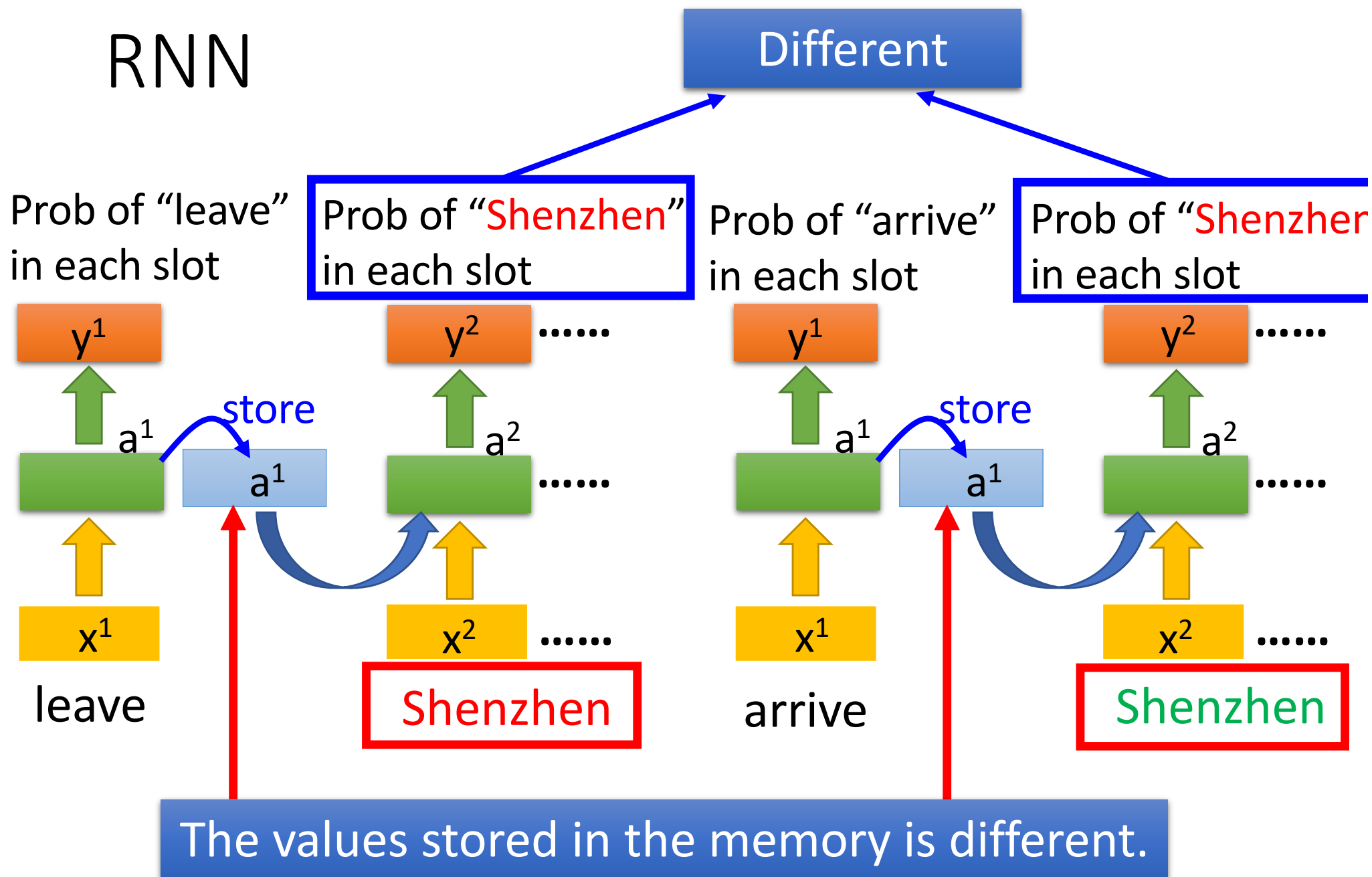
Probability of
“arrive” in each slot

Probability of “**Shenzhen**”
in each slot

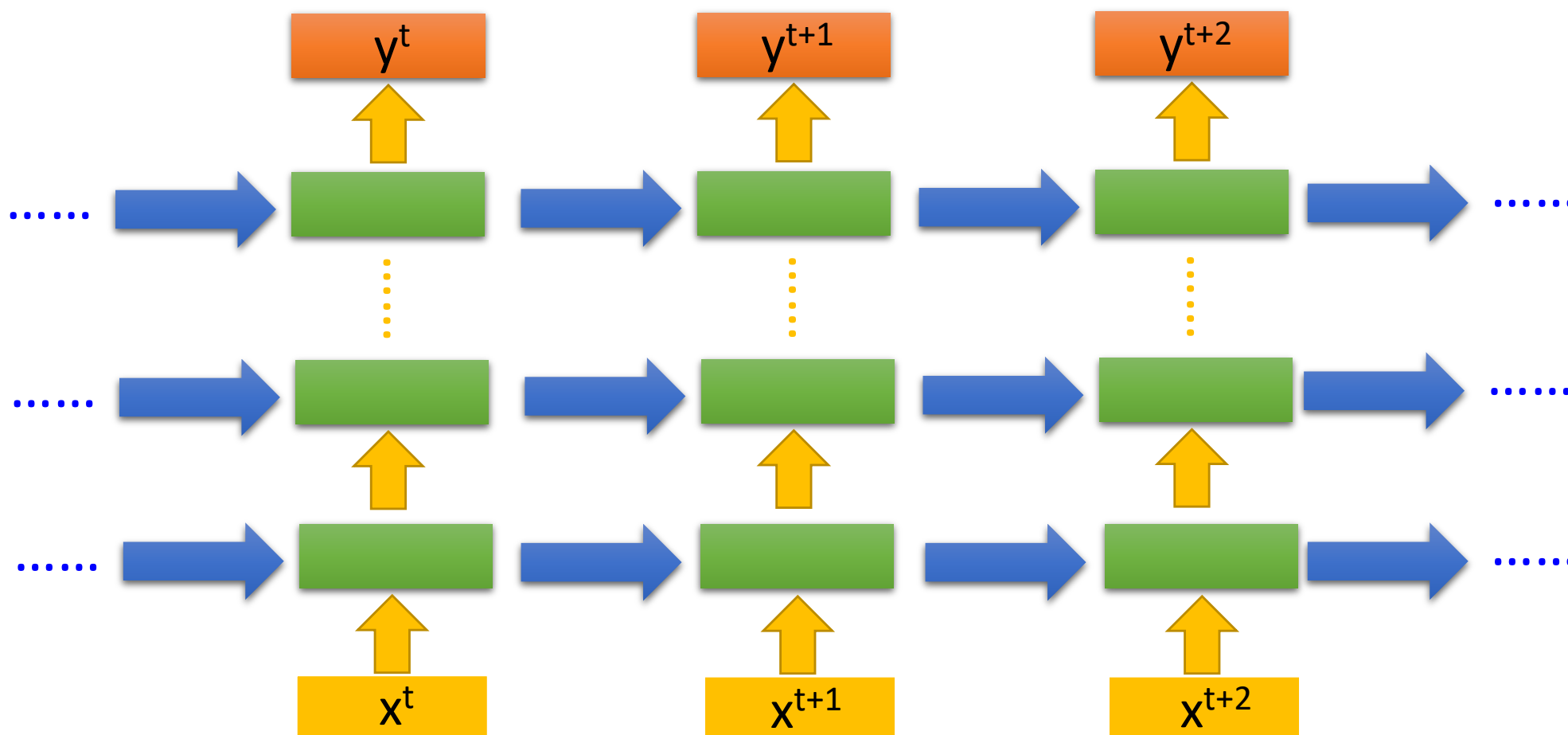
Probability of
“on” in each slot



RNN

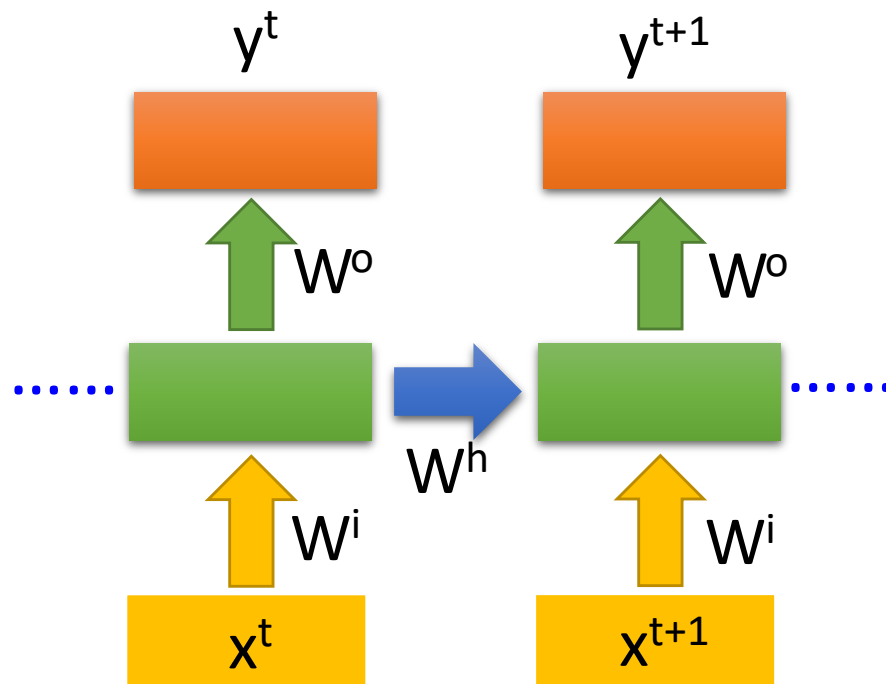


Of course it can be deep ...

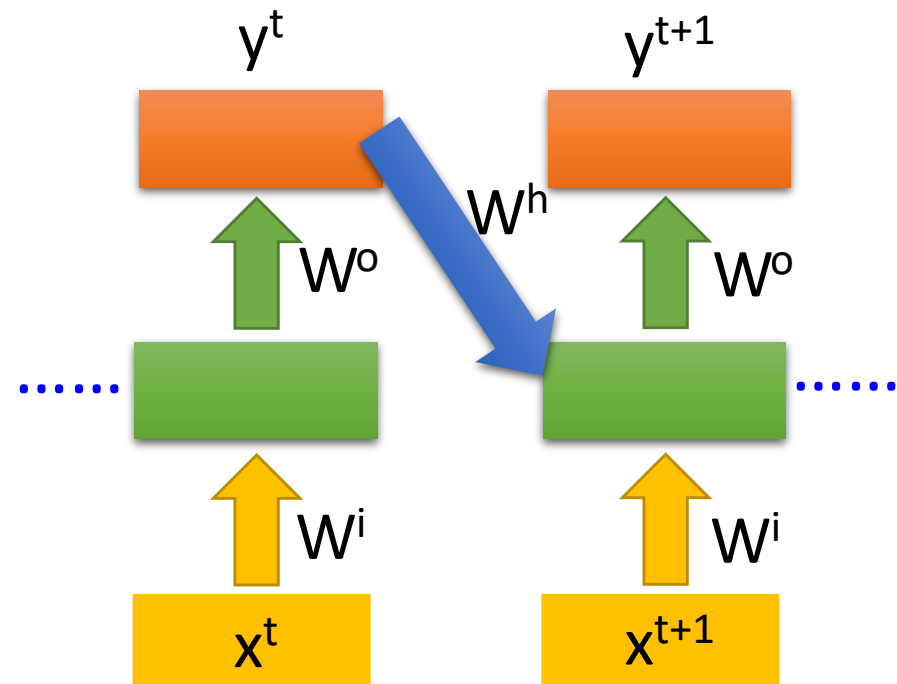


Elman Network & Jordan Network

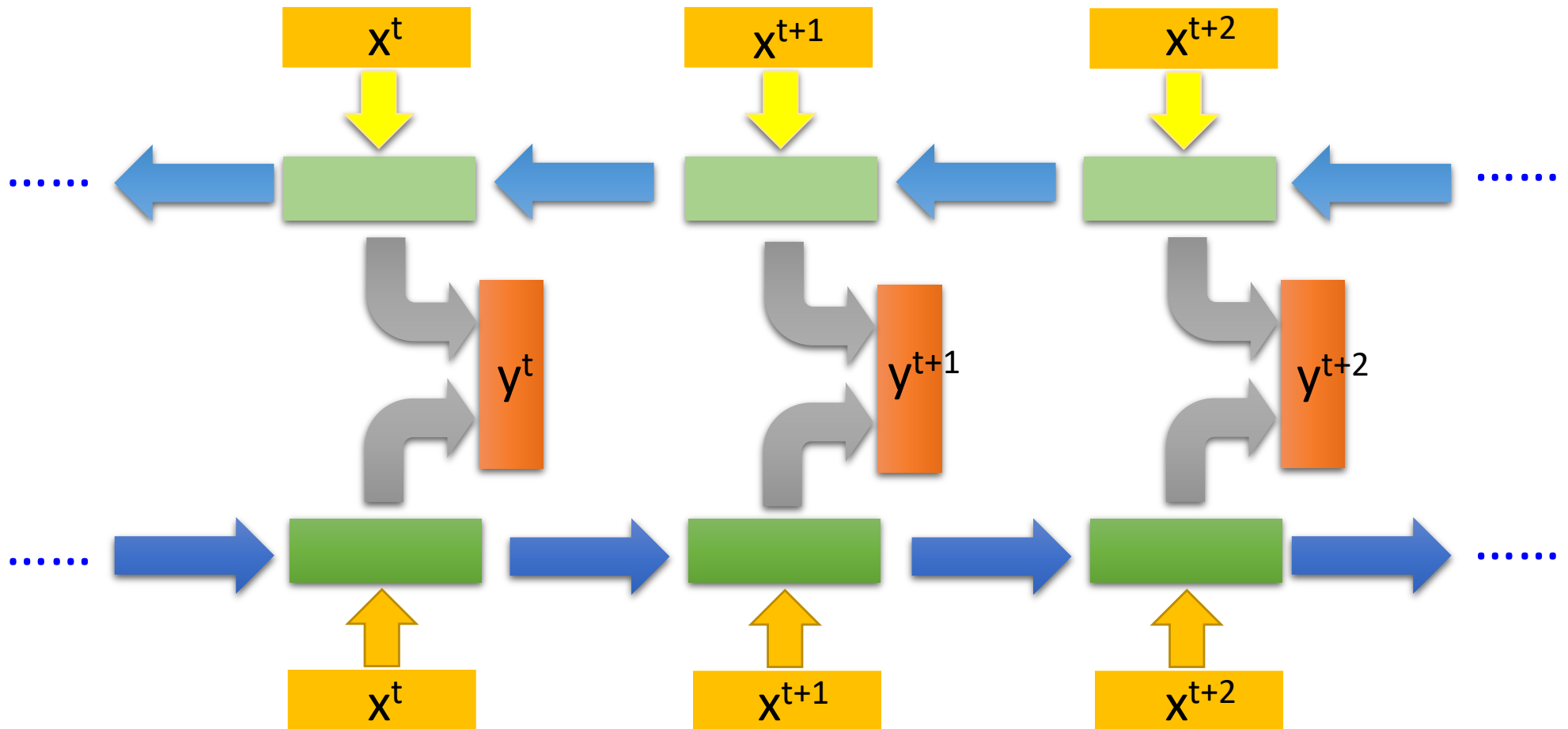
Elman Network



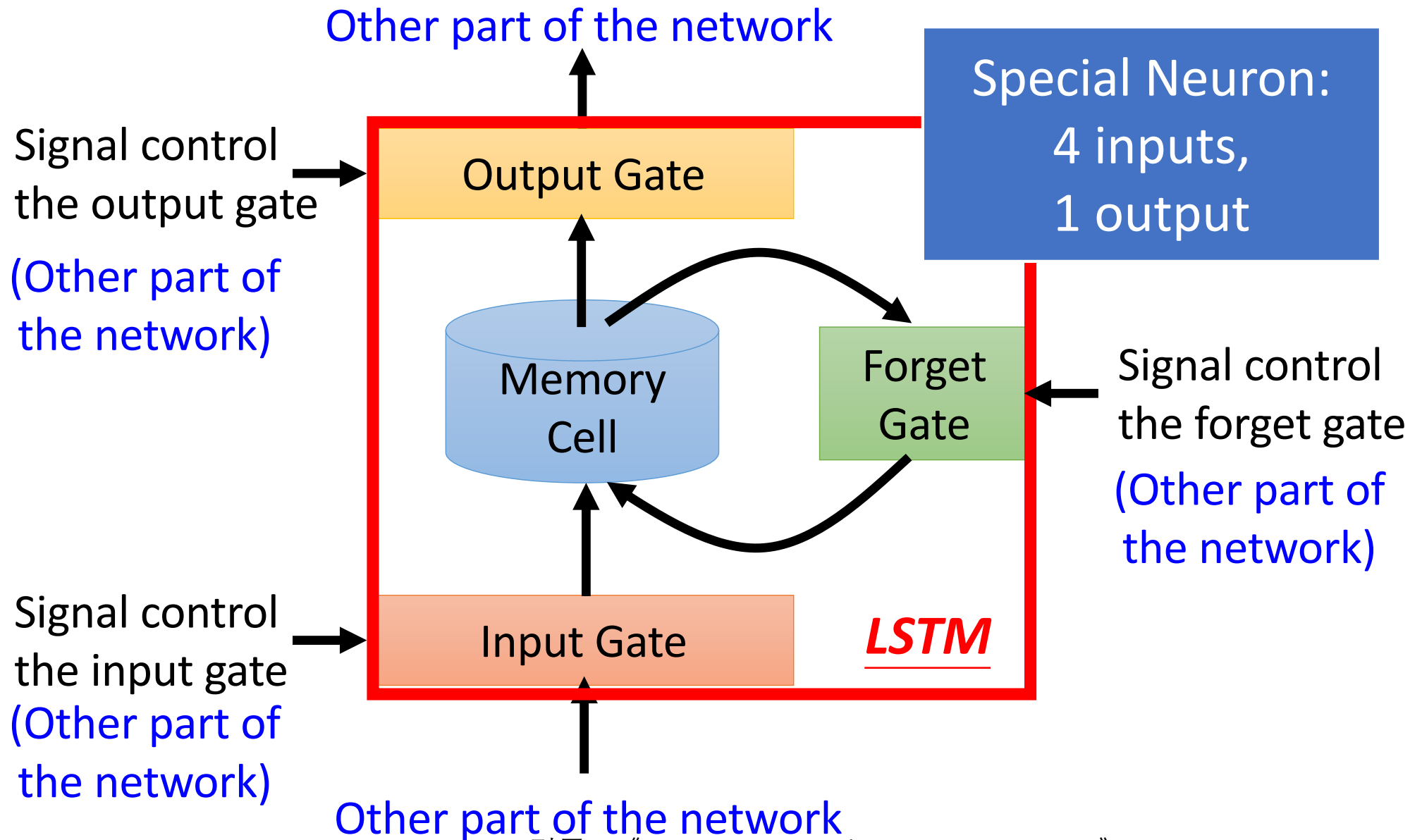
Jordan Network

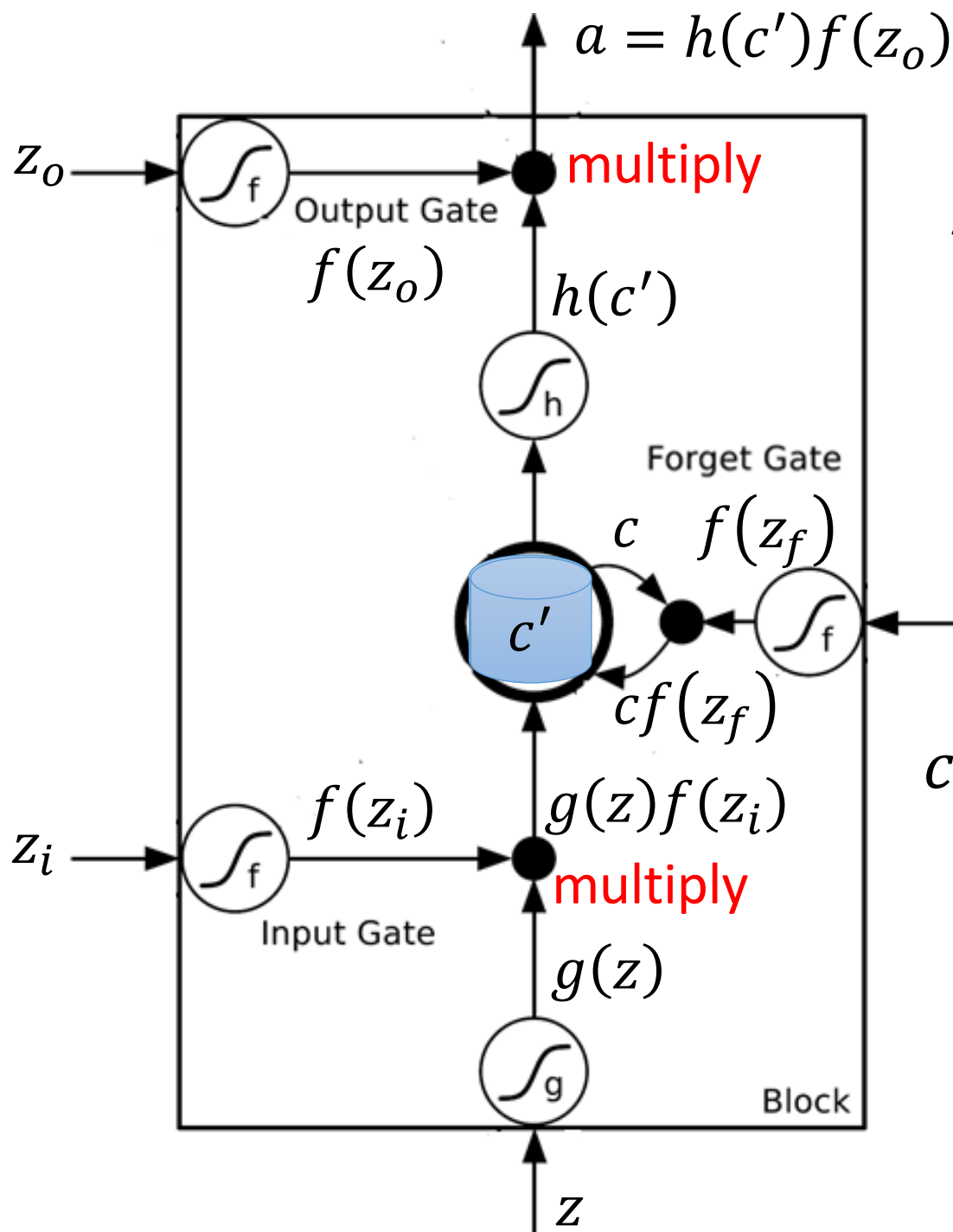


Bidirectional RNN



Long Short-term Memory (LSTM)





Activation function f is usually a sigmoid function

Between 0 and 1

Mimic open and close gate

$$c' = g(z)f(z_i) + cf(z_f)$$

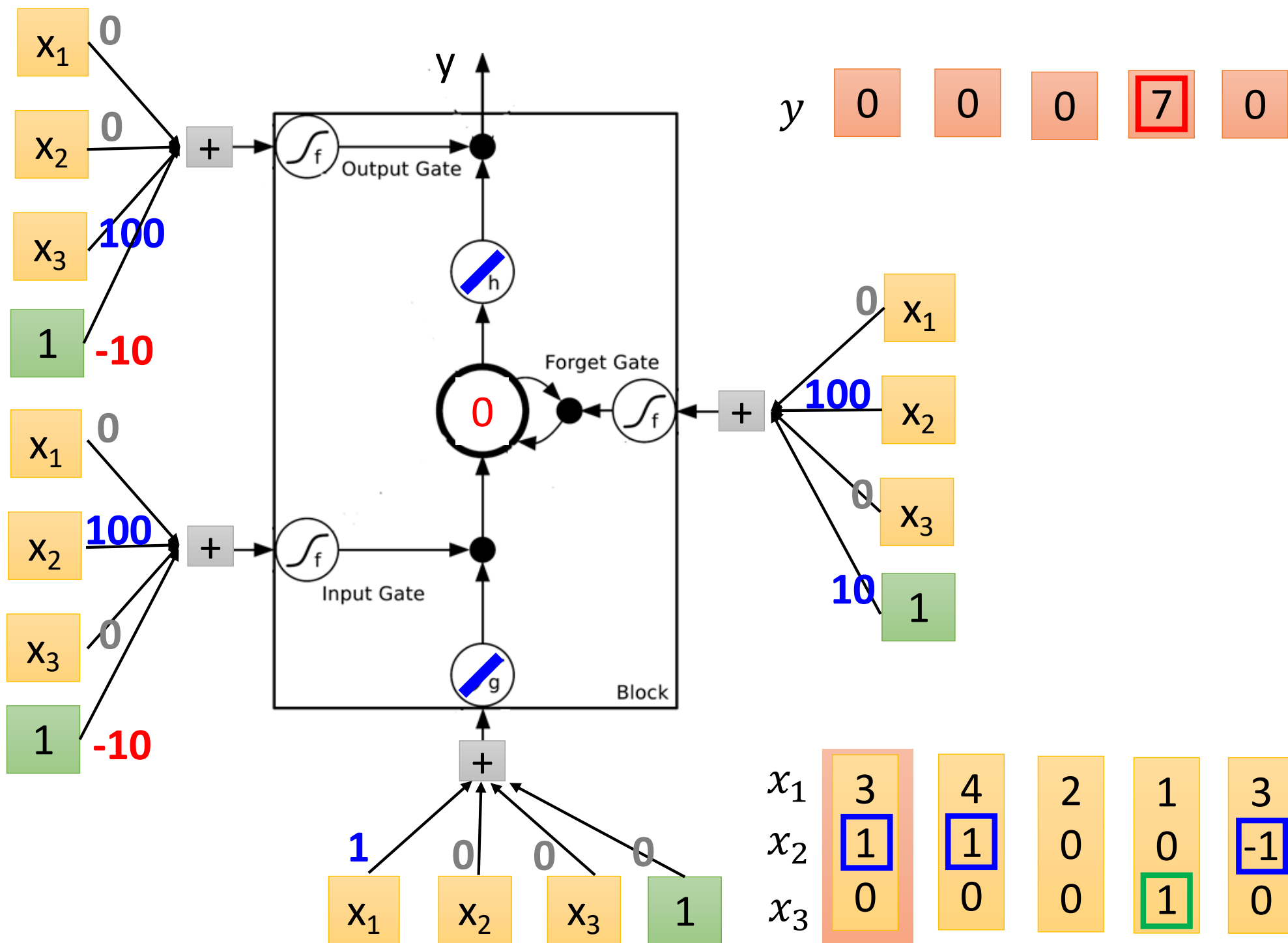
LSTM - 实例

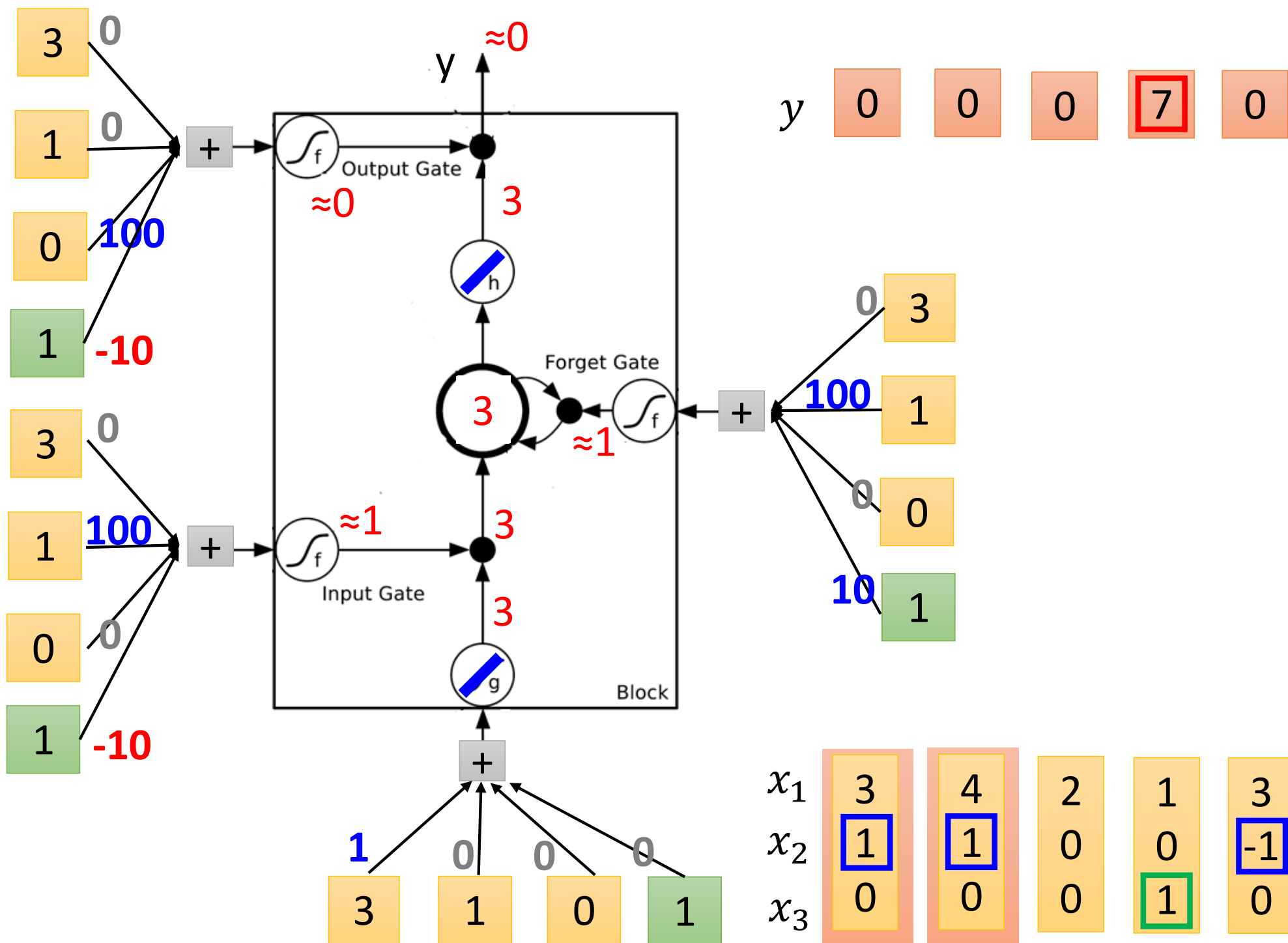
	0	0	3	3	7	7	7	0	6
x_1	1	3	2	4	2	1	3	6	1
x_2	0	1	0	1	0	0	-1	1	0
x_3	0	0	0	0	0	1	0	0	1
y	0	0	0	0	0	7	0	0	6

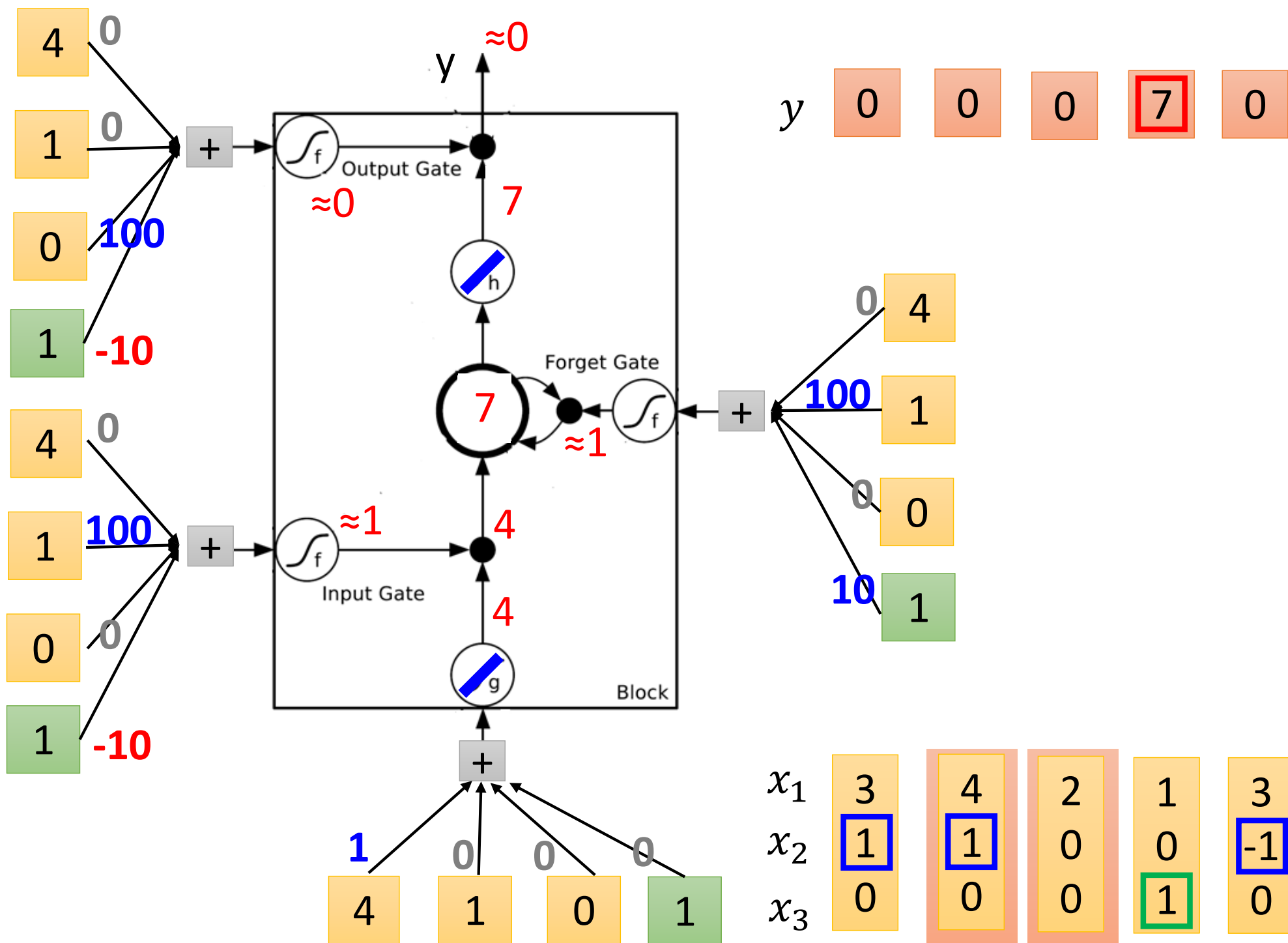
When $x_2 = 1$, add the numbers of x_1 into the memory

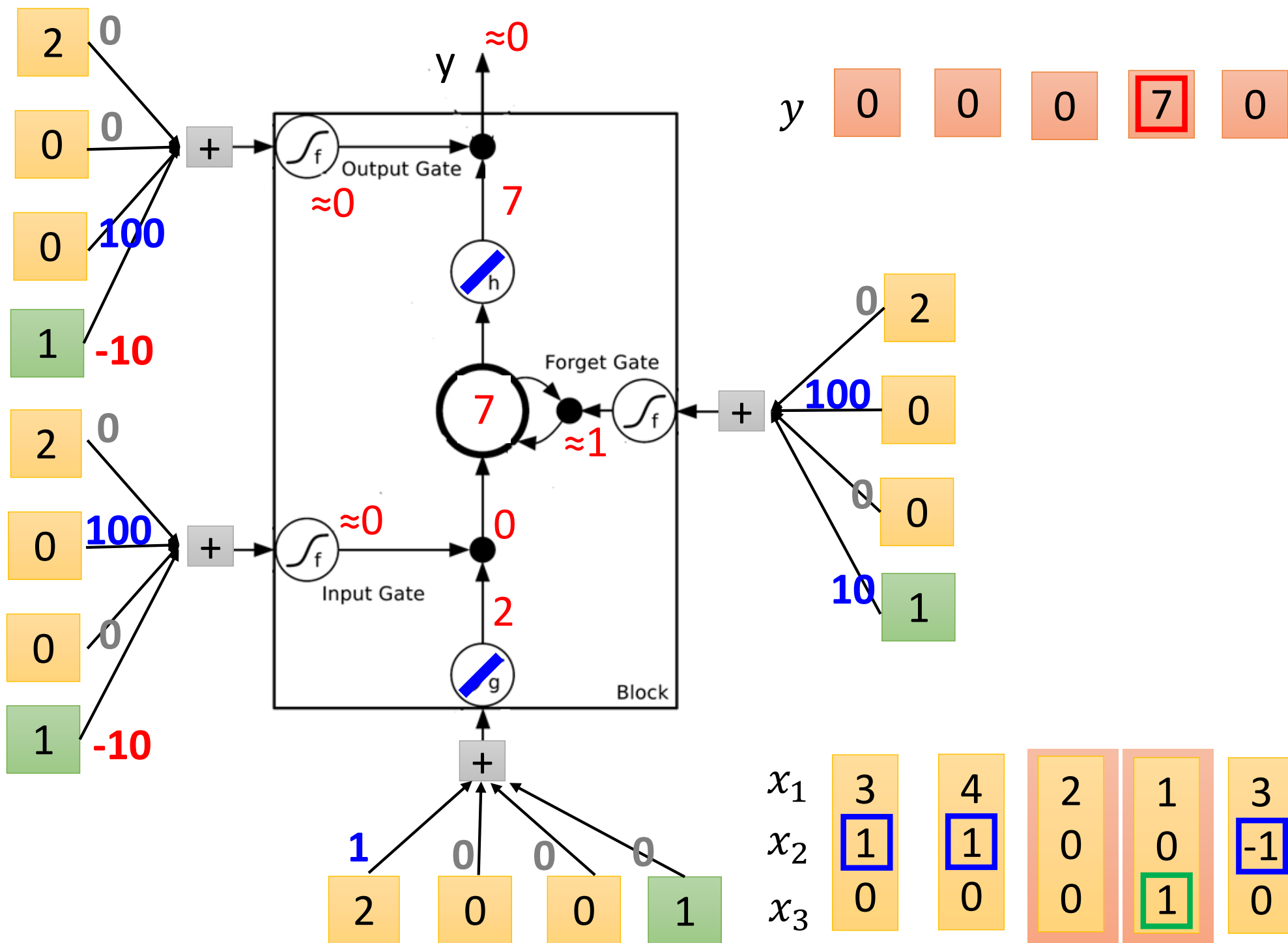
When $x_2 = -1$, reset the memory

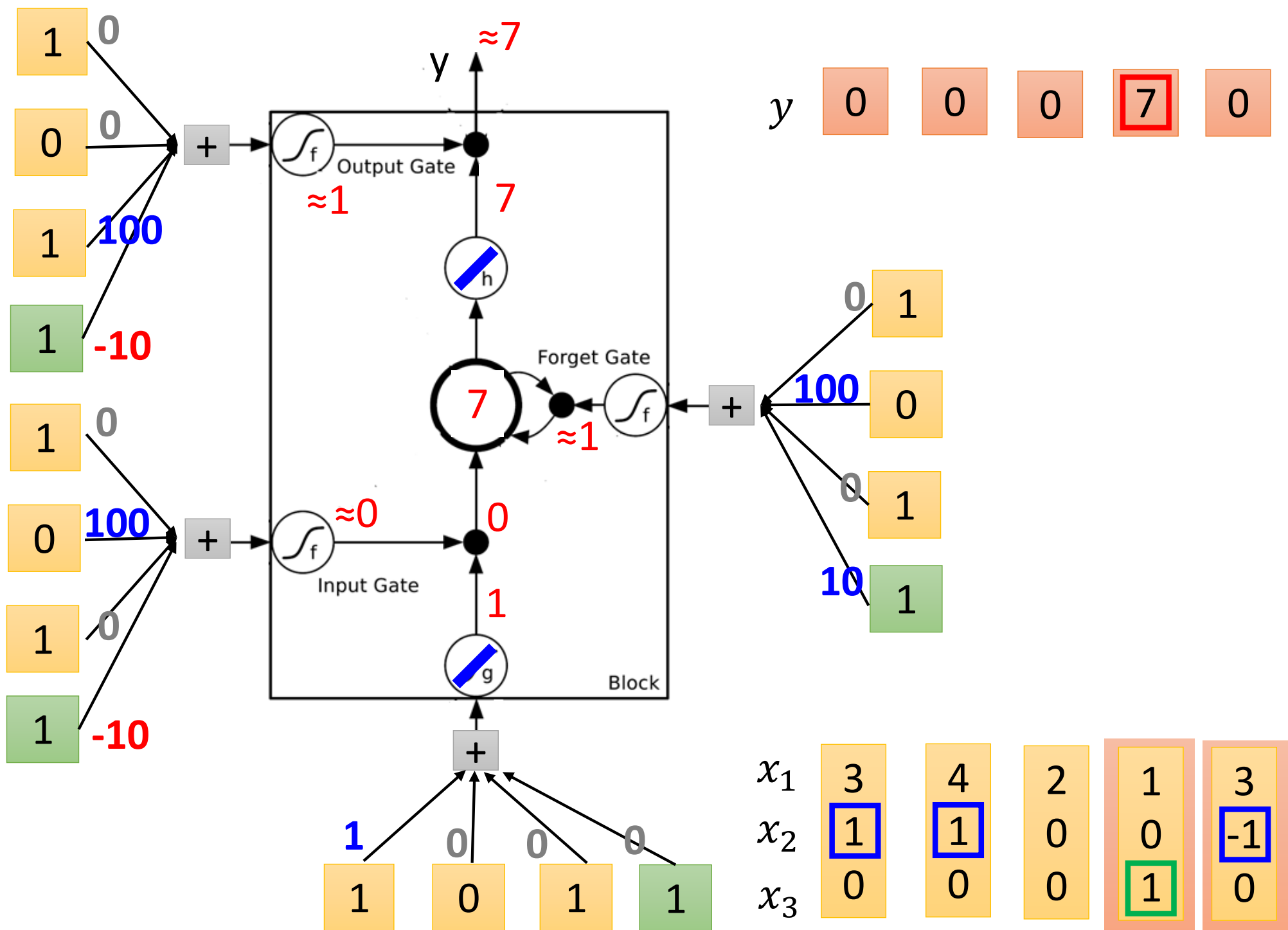
When $x_3 = 1$, output the number in the memory.

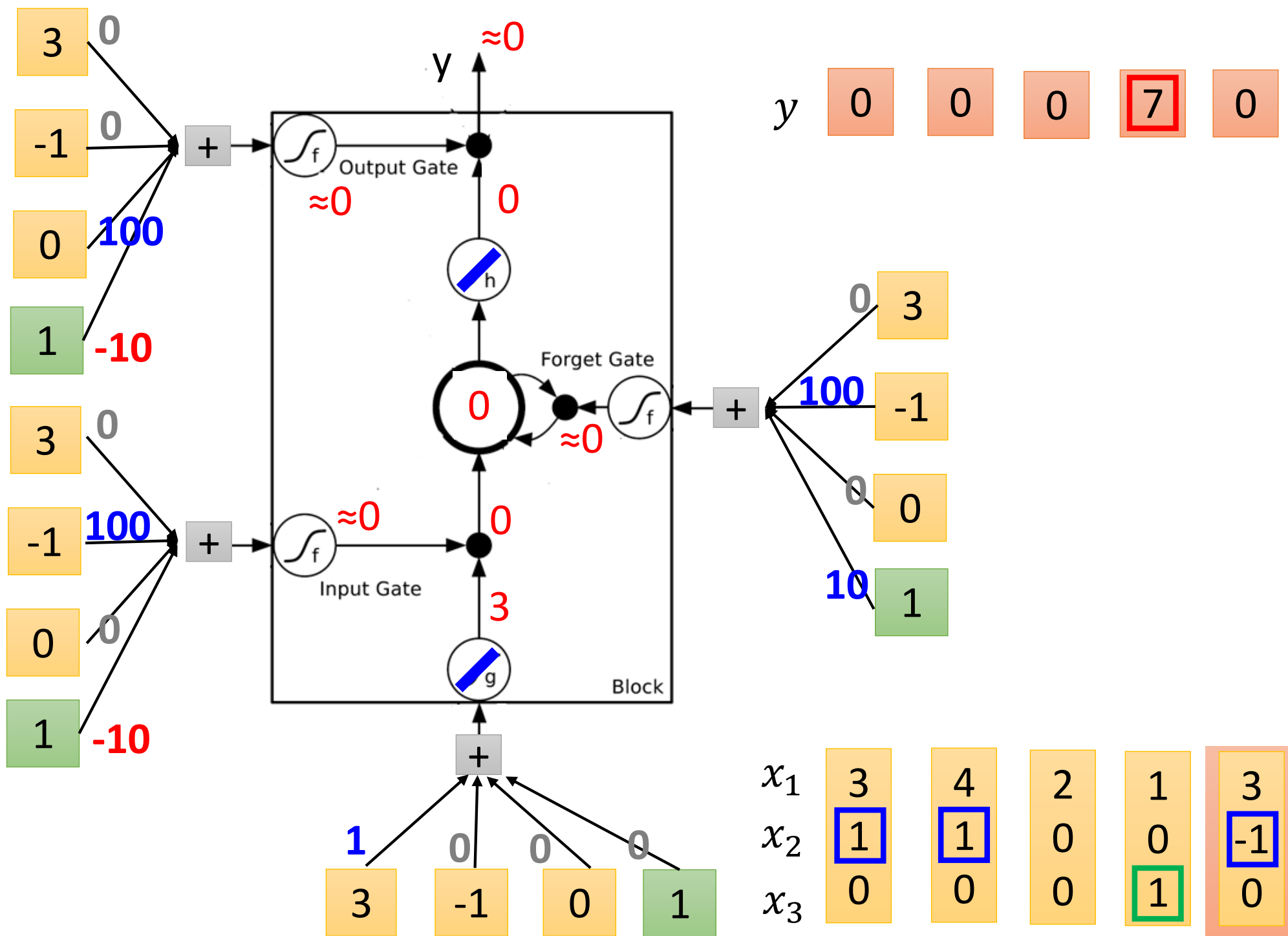






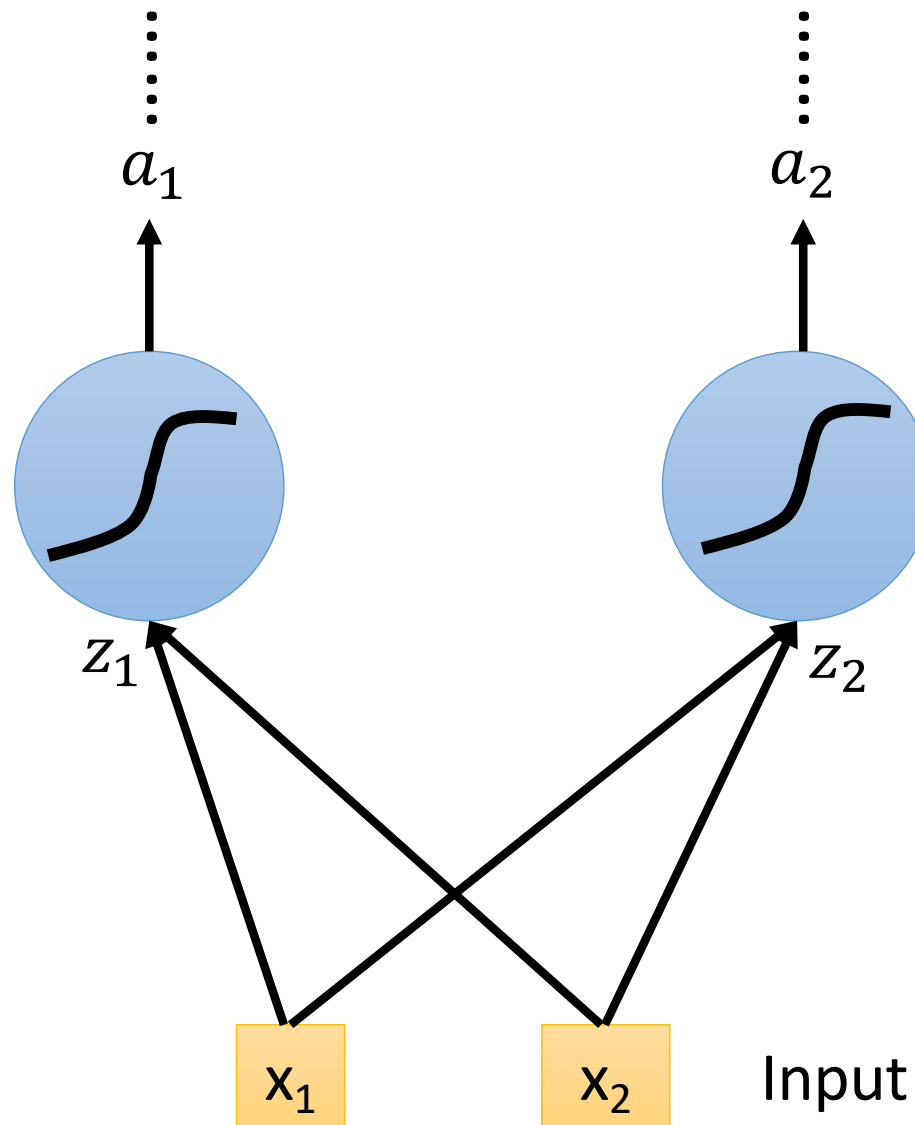


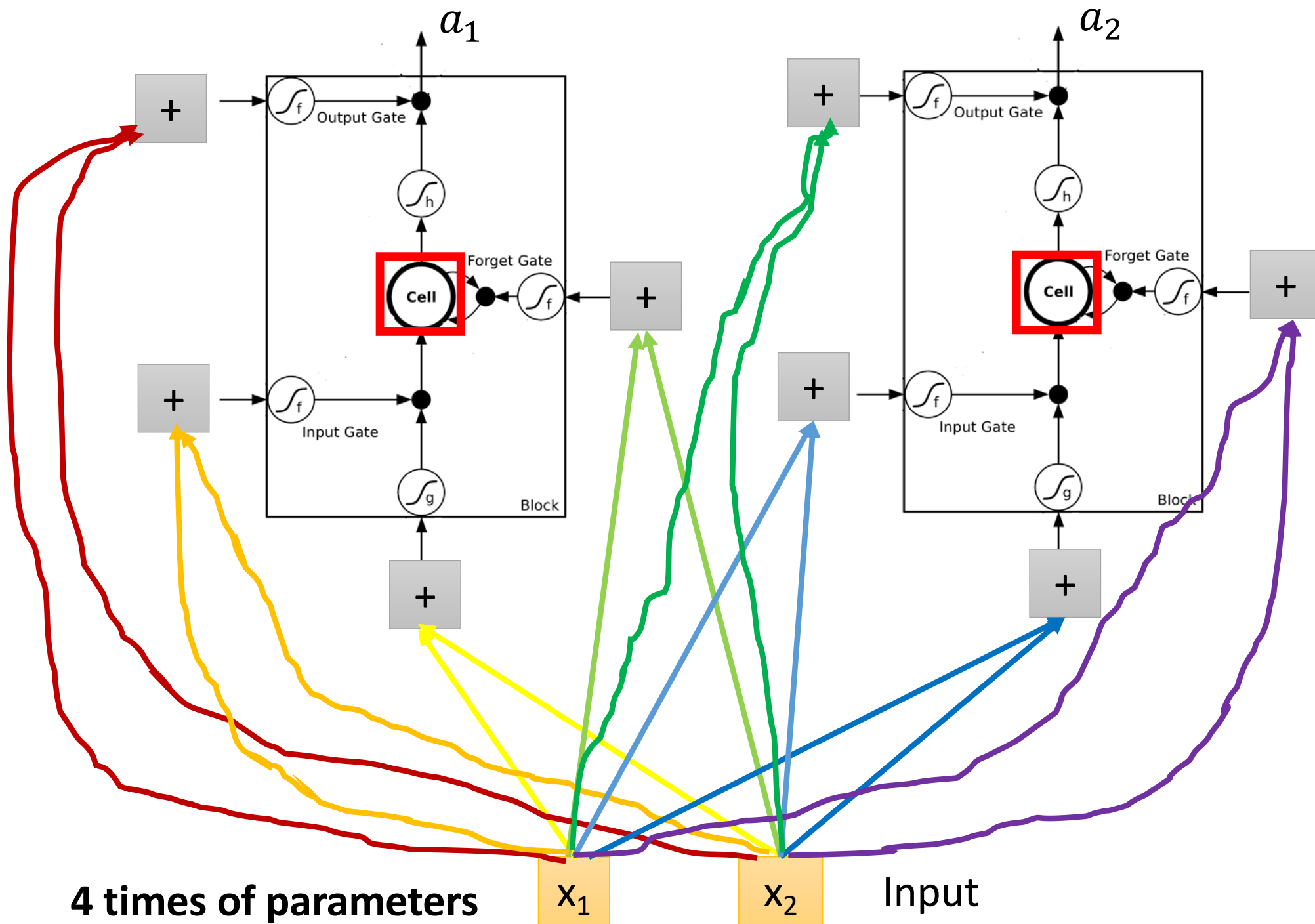




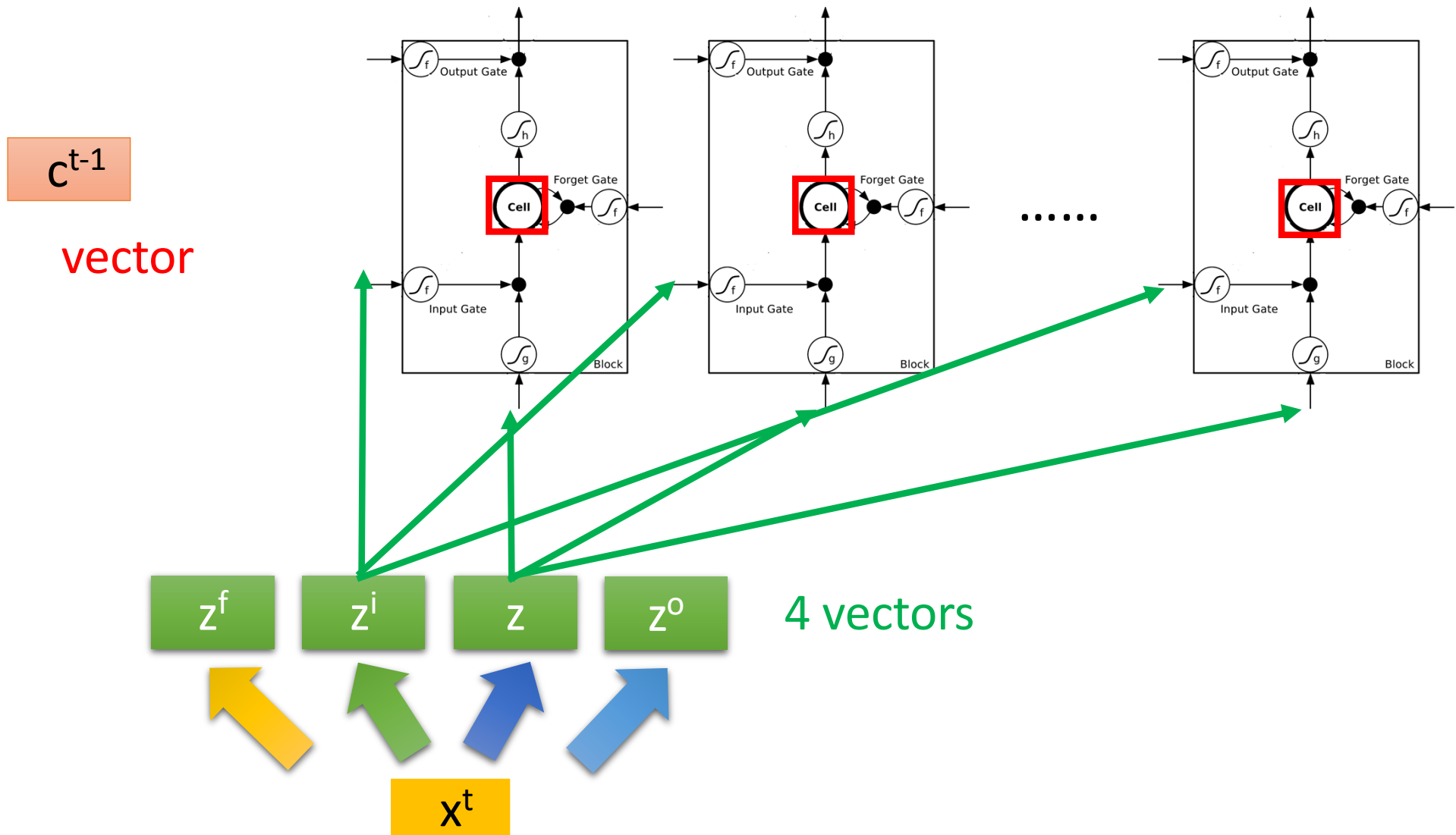
Original Network:

- Simply replace the neurons with LSTM

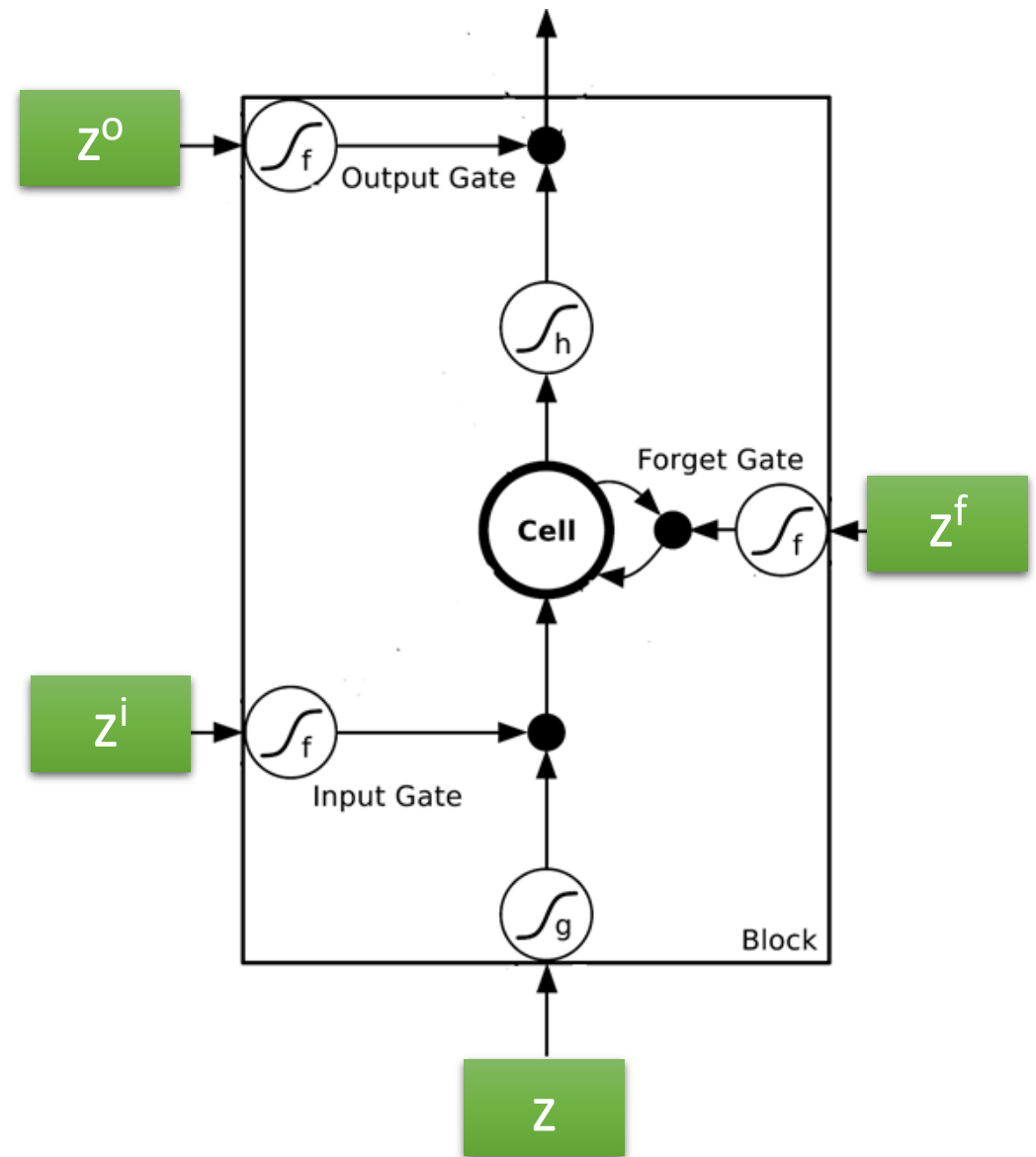
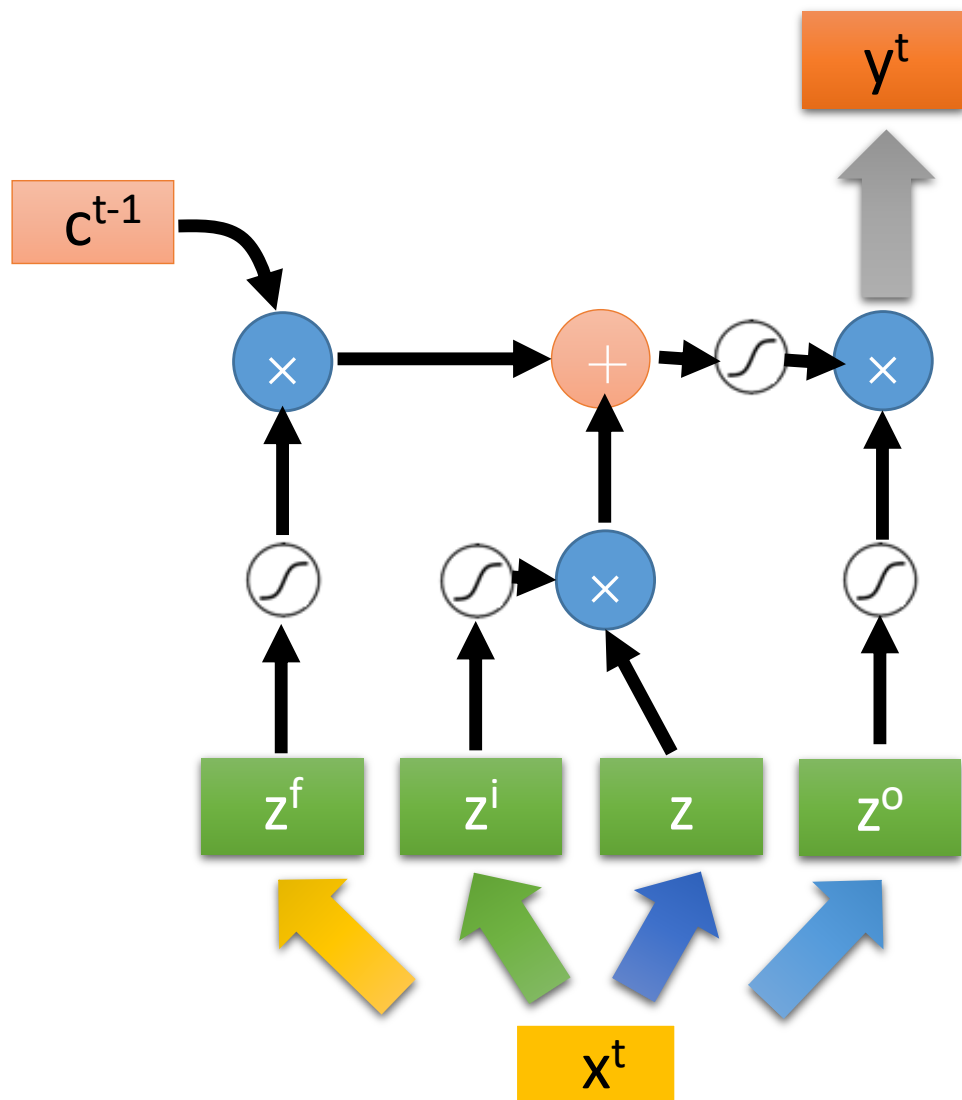




LSTM

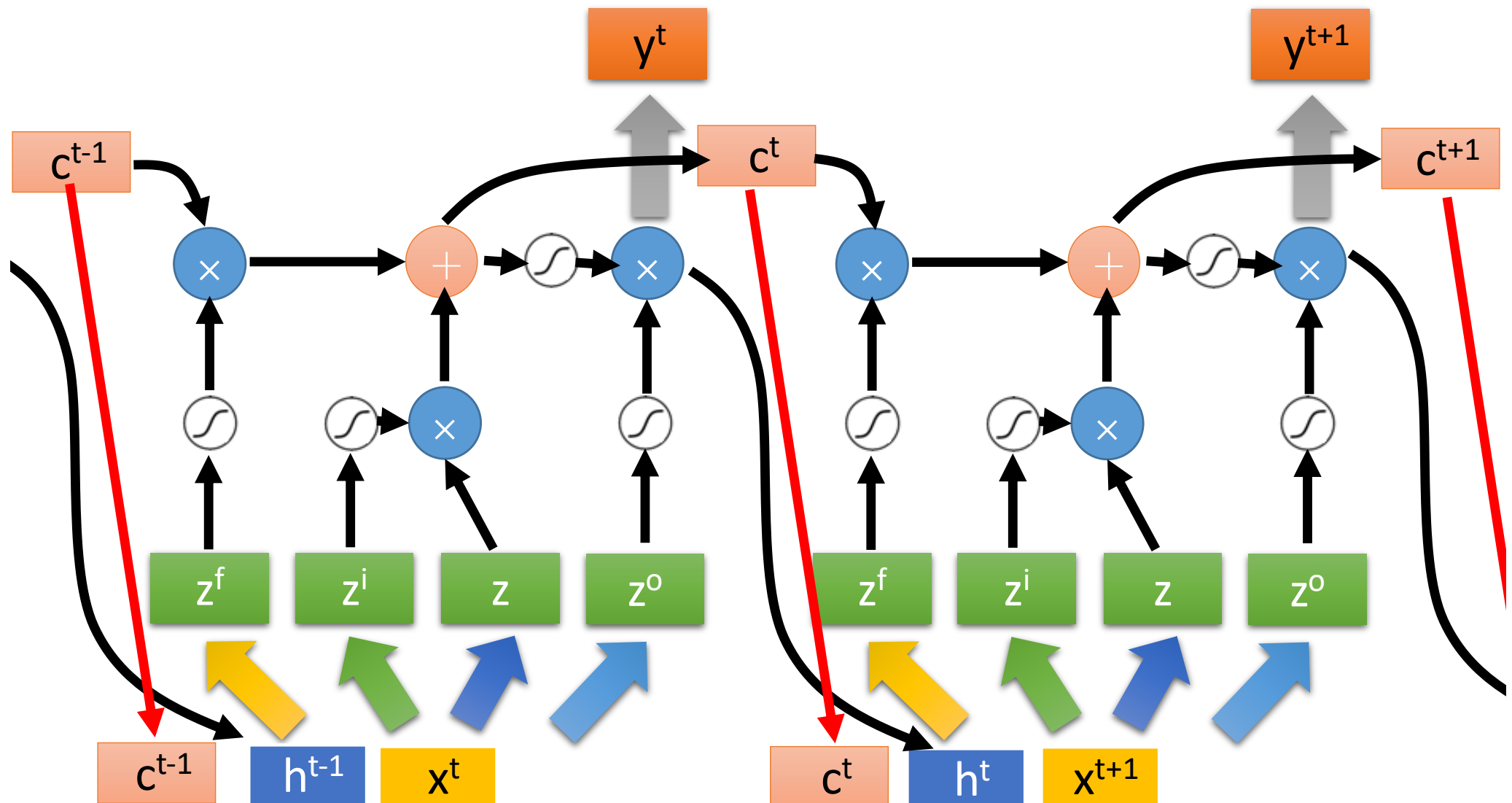


LSTM



LSTM

Extension: "peephole"



Multiple-layer LSTM

Don't worry if you cannot understand this.
Keras can handle it.

Keras supports
“LSTM”, “GRU”, “SimpleRNN” layers

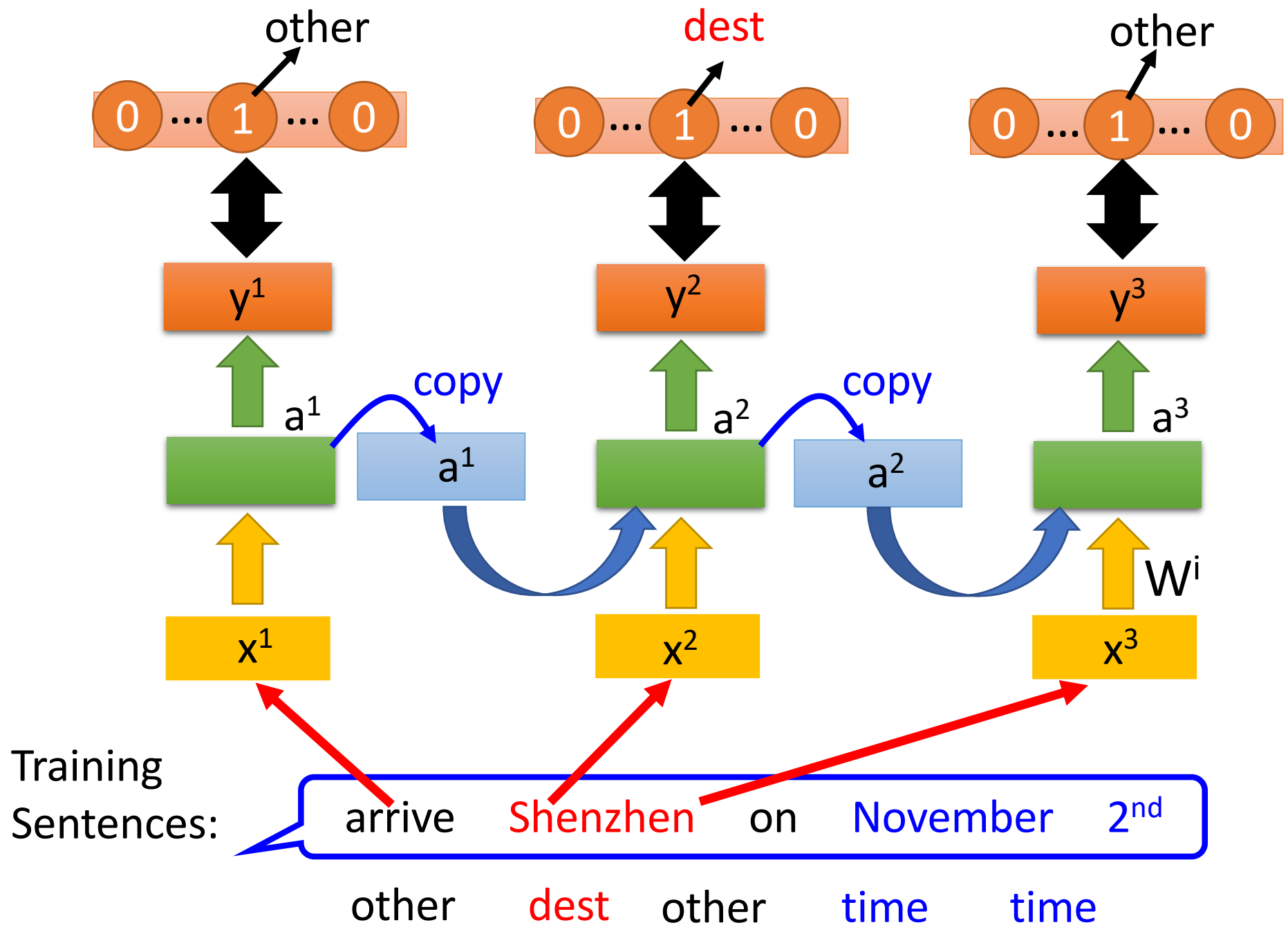
This is quite
standard now.

我到底看了什麼？

<https://img.komicolle.org/2015-09-20/src/14426967627131.gif>

引用：《Machine learning/Deep learning Course》，Hung-yi Lee

Learning Target



谢谢观看

